

Promises em .Js

#javascript

#programming

Video de ajuda

[Aprenda tudo sobre JavaScript Promises em 20 minutos](#)

Em JavaScript, uma **Promise** é um objeto que representa o resultado futuro de uma operação assíncrona.

Ela existe porque algumas tarefas levam tempo para terminar:

- requisições HTTP (`fetch`)
- leitura de arquivos
- acesso a banco de dados
- timers (`setTimeout`)

Em vez de bloquear o programa esperando o resultado, o JavaScript continua executando e a Promise informa quando a operação termina.

Estados de uma Promise

Uma Promise pode estar em um destes estados:

1. `pending`

A operação ainda está em andamento.

```
Promise { <pending> }
```

2. `fulfilled`

A operação foi concluída com sucesso.

```
Promise { "Dados recebidos" }
```

3. `rejected`

A operação falhou.

```
Promise { <rejected> Error }
```

Criando uma Promise

```
const promessa = new Promise((resolve, reject) => {  
  const sucesso = true;  
  if (sucesso) {  
    resolve("Operação concluída");  
  } else {  
    reject("Ocorreu um erro");  
  }  
});
```

Aqui:

- `resolve()` marca a Promise como concluída
- `reject()` marca a Promise como falha

Consumindo uma Promise

```
.then()
```

Executa quando a Promise é resolvida.

```
promessa.then(resultado => {  
  console.log(resultado);  
});
```

Saída:

```
Operação concluída
```

```
.catch()
```

Executa quando ocorre erro.

```
promessa .then(resultado => {  
  console.log(resultado);  
})  
.catch(erro => {  
  console.log(erro);  
});
```

Exemplo com `setTimeout`

```
const promessa = new Promise(resolve => {  
  setTimeout(() => {  
    resolve("Página carregada");  
  }, 3000);  
});  
promessa.then(msg => {  
  console.log(msg);  
});
```

Fluxo:

```
Promise criada  
↓  
3 segundos  
↓  
resolve("Página carregada")  
↓  
.then() executa
```

Encadeamento

O retorno de um `.then()` vai para o próximo:

```
Promise.resolve(5)  
.then(num => num * 2)  
.then(num => num + 1)  
.then(resultado => console.log(resultado));
```

Saída:

`fetch()` retorna uma Promise

Por isso você já viu algo assim:

```
fetch("https://api.github.com/users/octocat")
  .then(response => response.json())
  .then(data => console.log(data));
```

O JavaScript não sabe quando a resposta chegará, então usa Promises.

Relação com callbacks

Antes:

```
carregarPagina(() => console.log("Pronto"));
```

Com Promises:

```
carregarPagina().then(() => console.log("Pronto"));
```

Promises ajudam a evitar o chamado *callback hell*:

```
a(() => {
  b(() => {
    c(() => {
      d(() => {});
    });
  });
});
```

Resumo

- Promise representa um resultado futuro
- possui os estados `pending`, `fulfilled` e `rejected`

- `resolve()` indica sucesso
- `reject()` indica falha
- `.then()` trata sucesso
- `.catch()` trata erros
- é a base do `fetch()` e do `async/await` moderno em JavaScript